

SQL Agent with LangChain

A hands-on project exploring how to connect Large Language Models to relational databases — starting from raw `sqlite3`, moving through `SQLAlchemy` as a universal database connector, and finally building a working **SQL Agent** using LangChain that can answer natural language questions by generating, executing, and explaining SQL queries. The live app lets anyone ask a question in plain English — like "Who is Bob Brown's manager?" — and get a real, accurate answer back, with no SQL knowledge required.

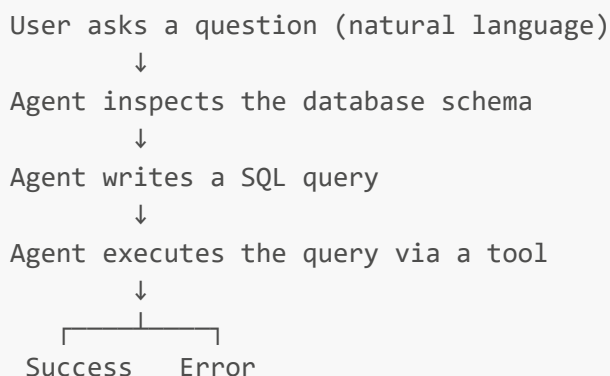
Overview

This project walks through the full stack needed to build a database-aware AI agent:

1. **SQLite3 fundamentals** — connecting, creating tables, inserting data, and querying with Python's built-in `sqlite3` module.
2. **SQLAlchemy** — a universal database toolkit that works across SQLite, MySQL, PostgreSQL, and more, using the same consistent API.
3. **MySQL integration** — connecting to a production-style, cloud-hosted MySQL database instead of a local file-based database.
4. **LangChain SQL Agent (toolkit-based)** — using LangChain's built-in `SQLDatabaseToolkit` to auto-generate the tools an agent needs to interact with a database.
5. **Manual Toolkit creation** — building the same set of database tools from scratch using LangChain's `@tool` decorator, to understand what the toolkit does under the hood.

How the Live App Works

1. You type a question.
2. The question goes to Google's Gemini model, along with four tools it can choose to use: list tables, inspect a table's schema, run a SQL query, and validate a query before running it.
3. The model decides, on its own, which tools to use and in what order — checking the schema before writing a query, and retrying if a query fails.
4. The final answer is shown in plain English, with a "show your work" section revealing every step the agent took along the way.



Agent fixes the query and retries
↓
Agent explains the result in plain language

Why SQLite First, Then MySQL?

SQLite requires no server setup — it's a single file, making it ideal for quickly prototyping and testing an agent's logic in the notebook. Once the agent's reasoning is verified, the same code can point to a production database by simply swapping the connection string, since both [SQLAlchemy](#) and LangChain's [SQLDatabase](#) utility are database-agnostic. The deployed app connects to a hosted MySQL instance (Aiven), not SQLite.

Project Structure

```
.
├── sqlapp.py                # The live Streamlit application
├── sql_agent.ipynb          # Exploratory notebook – full walkthrough &
manual toolkit build
├── html_sql_agent1.py       # HTML flow diagram: SQLite3 workflow
├── sql_alchemy_html_flow.py # HTML flow diagram: SQLAlchemy workflow
├── html_langchain_agent_flow.py # HTML flow diagram: LangChain SQL Agent
workflow
├── requirements.txt
├── .env.example             # Template for required environment variables
├── .env                    # Real credentials – not committed
├── ca.pem                  # Aiven MySQL SSL certificate – not committed
├── .streamlit/config.toml  # App theme configuration
└── .gitignore
```

Sample Dataset

The [EMPLOYEES](#) table represents an organizational hierarchy:

Column	Type	Description
EMPLOYEE_ID	INTEGER	Unique employee identifier
NAME	TEXT	Employee name
MANAGER_ID	INTEGER	ID of the employee's manager (nullable)
MANAGER_NAME	TEXT	Name of the employee's manager (nullable)

This same schema is explored in SQLite and MySQL in the notebook, and lives in a hosted MySQL (Aiven) instance for the live app.

Key Concepts Covered

- **Connect → Cursor → Create → Insert → Commit → Query → Close**: the core SQLite3 workflow, including why `commit()` is required before `close()` to persist changes.
- **SQLAlchemy as a universal connector**: one consistent API (`create_engine`, `text()`, `.connect()`) that works across different database engines by simply changing the connection string.
- **Safe credential handling**: using `urllib.parse.quote_plus()` to URL-encode passwords containing special characters (e.g., `@`) so they don't break the connection string.
- **LangChain's `SQLDatabase` utility**: wrapping a SQLAlchemy engine so an LLM can inspect table names and schemas before writing queries.
- **`SQLDatabaseToolkit`**: LangChain's built-in toolkit that auto-generates four tools — list tables, get schema, run a query, and check query syntax — from a single `SQLDatabase` object.
- **`create_agent`**: wiring an LLM and the SQL tools together into an agent capable of multi-step reasoning (check schema → write query → execute → self-correct on error → explain result).
- **Manual tool creation**: rebuilding the toolkit's core functionality by hand using the `@tool` decorator and raw SQLAlchemy calls, to understand exactly what the toolkit abstracts away.
- **Provider-agnostic model swapping**: LangChain's standardized model interface meant switching the live app from OpenAI to Google Gemini was a two-line code change, not a rewrite.
- **Secure hosted database connections**: connecting to Aiven's managed MySQL over SSL using a downloaded CA certificate, rather than an unauthenticated local connection.

Setup

1. Install dependencies

```
pip install -r requirements.txt
```

2. Set up your environment variables

Copy `.env.example` to `.env` and fill in your real values:

```
MYSQL_USER=your_mysql_user
MYSQL_PASSWORD=your_mysql_password
MYSQL_HOST=your_mysql_host
MYSQL_PORT=your_mysql_port
MYSQL_DATABASE=your_database_name
GOOGLE_API_KEY=your_gemini_api_key
```

3. Get a free Gemini API key

Visit aistudio.google.com, sign in, and generate an API key — no credit card required.

4. Set up your MySQL database

This project uses Aiven's free MySQL tier. Aiven requires SSL — download your service's CA certificate, save it as `ca.pem` in the project root, and create

the `EMPLOYEES` table using the schema above.

5. Run the app

```
streamlit run sqlapp.py
```

Or, explore the build process: open `sql_agent.ipynb` and run the cells sequentially — it's structured as a progressive walkthrough, from raw SQLite3 to a fully working LangChain SQL Agent, including the manual toolkit build.

Example Query

```
question = (
    "Tell me the manager's name for each employee along with their employee ID "
    "and manager ID. If no manager exists, return 'CEO' as the manager name "
    "and 0 as the manager ID."
)

answer = sql_agent.invoke({'messages': [{'role': 'user', 'content':
question}]})
print(answer['messages'][-1].content)
```

The agent inspects the schema, writes an appropriate `LEFT JOIN` query, executes it, and returns a natural-language, tabular-formatted answer.

Tech Stack

- **Python 3.12**
- **Streamlit** — the live web UI
- **LangChain** (`langchain.agents.create_agent`) — the agent loop and tool-calling
- **LangChain Google GenAI** (`ChatGoogleGenerativeAI`) — connects the agent to Gemini
- **SQLAlchemy**
- **SQLite3** (built-in, notebook only)
- **PyMySQL** (MySQL driver)
- **python-dotenv**
- **Aiven for MySQL** — hosted database (SSL-only connection)

Notes

- The `SQLDatabaseToolkit` internally relies on SQLAlchemy — using the toolkit doesn't remove the need for SQLAlchemy, it simply automates the tool-writing step.
- A **chat model** (not a plain completion model) must be used with `create_agent`, since tool-calling support requires it — the live app uses `ChatGoogleGenerativeAI`.

- Passwords containing special characters (like @) must be URL-encoded with `quote_plus()` before being placed in a connection string, otherwise the connection parser misreads the credentials.
- Aiven requires SSL for all connections — the connection string includes `?ssl_ca=ca.pem`, pointing at a locally downloaded (and gitignored) certificate.

Development Notes

This project went through a real cleanup and hardening pass after the initial draft, working through each issue methodically rather than shipping the first working version:

- **Dependency cleanup:** the original `requirements.txt` was a 415-package freeze of an unrelated ML environment (torch, transformers, spacy, etc.), saved in the wrong encoding (UTF-16 instead of UTF-8) and containing Windows-only packages that would fail to install on any Linux deploy target. Rebuilt from a clean, project-scoped virtual environment.
- **Fixed a real config bug:** the app told users to set `MYSQL_DATABASE` in their `.env`, but the code silently ignored it and used a hardcoded value. Fixed to actually read the environment variable.
- **Flattened the repo structure:** the project originally lived inside a redundant nested folder from an extracted zip. Restructured to a clean root layout.
- **Switched from OpenAI to Google Gemini:** to keep this project genuinely free to run and demo (no risk of an unexpected bill on a public-facing app), swapped `ChatOpenAI` for `ChatGoogleGenerativeAI` — a two-line change thanks to LangChain's standardized model interface.
- **Added session-based usage limits:** since this runs as a public demo, a per-session question cap prevents any single visitor from exhausting the shared API quota.
- **Wired up a real hosted database:** connected to Aiven's managed MySQL with proper SSL certificate verification, replacing local-only testing.
- **Custom visual design:** restyled the default Streamlit theme with custom CSS rather than shipping the default look.

Note: a chunk of this work was done offline (local development and testing without pushing incrementally), which is why some of this shows up as fewer, larger commits rather than a granular step-by-step history.

Roadmap / Future Scope

- **Guardrails against destructive queries:** connect the app through a MySQL user with read-only (`SELECT`-only) grants, plus an app-level check that rejects any non-`SELECT` query before it reaches the database — enforced outside the AI's control, not just requested via prompt.
- **Authentication:** a login screen (via `streamlit-authenticator` or similar) so the app knows who's asking, rather than treating every visitor identically.

- **True role-based access control:** map each authenticated user's role to its own MySQL user with distinct **GRANT** permissions — e.g. an HR role that can see salary data, and a general role that can't — enforced at the database level, not the system prompt, since prompt-based restrictions can be bypassed by a sufficiently crafted question.
- **Permission-aware UI:** show each logged-in user which tables/columns they currently have access to, based on their real database grants.
- **Deployment:** Streamlit Community Cloud, with the SSL certificate and credentials handled through the platform's secrets manager rather than a local file.